# Création d'un capteur connecté

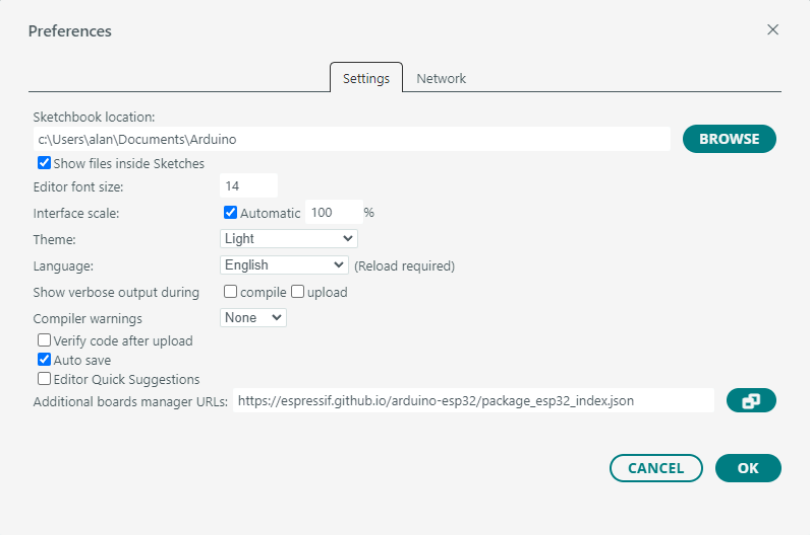
Mise en route

## 0. OBJECTIF

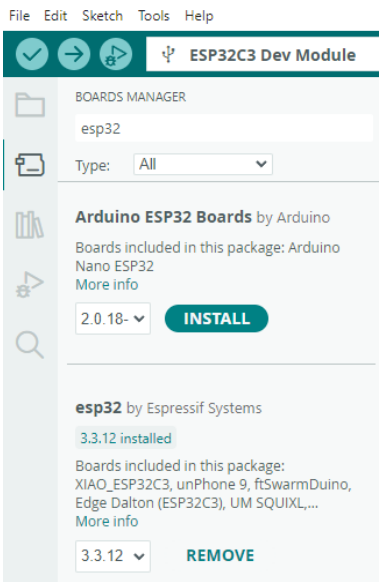
- votre ESP32 publie une donnée à intervalle régulier
- votre tuile IHM affiche cette donnée
- un bouton de votre tuile IHM envoie une commande au capteur
- votre capteur reçoit cette commande

## 1. INSTALLATION DE LA BIBLIOTHÈQUE

1. Dans l'Arduino IDE : File > Preferences : ajoutez le repo des cartes ESP32 aux gestionnaires de cartes additionnels : *https://espressif.github.io/arduino-esp32/package_esp32_index.json*



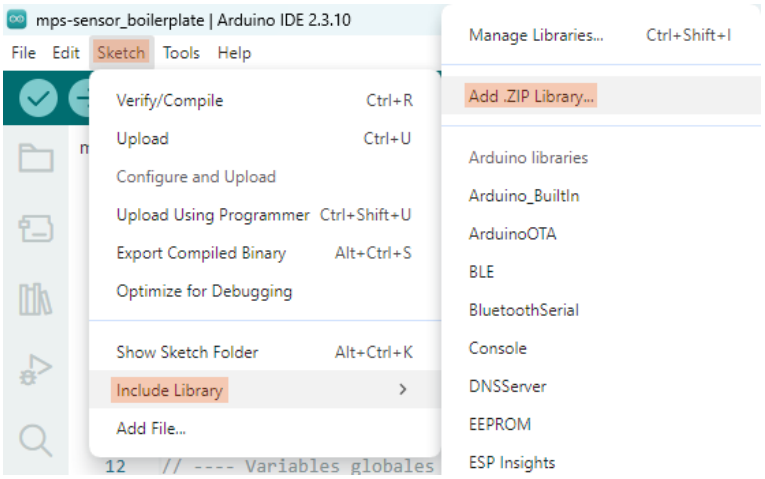

2. Dans l'onglet "BOARDS MANAGER", installer le paquet des cartes "esp32 by Espressif Systems" :



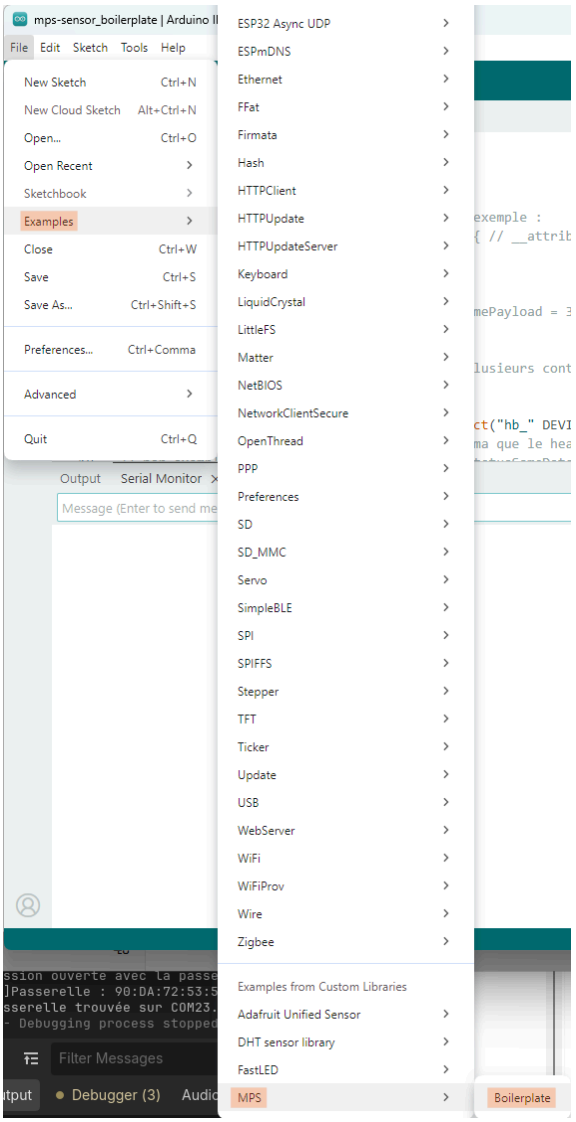
# Création d’un capteur connecté

Mise en route

3. Sketch > Include Library > Add .ZIP Library, sélectionnez “MPS.zip” dans le dossier de travail :




4. File > Examples > MPS > Boilerplate : Ouvrez-le et enregistrez-le sous un nouveau nom, il s’agit de la base de votre croquis.



# Création d’un capteur connecté

Mise en route

## 2. PARAMETRAGE DE CONFIG.H

Connectez la passerelle de test à votre PC et lancer l’IHM depuis Godot, la section “Output” doit afficher :

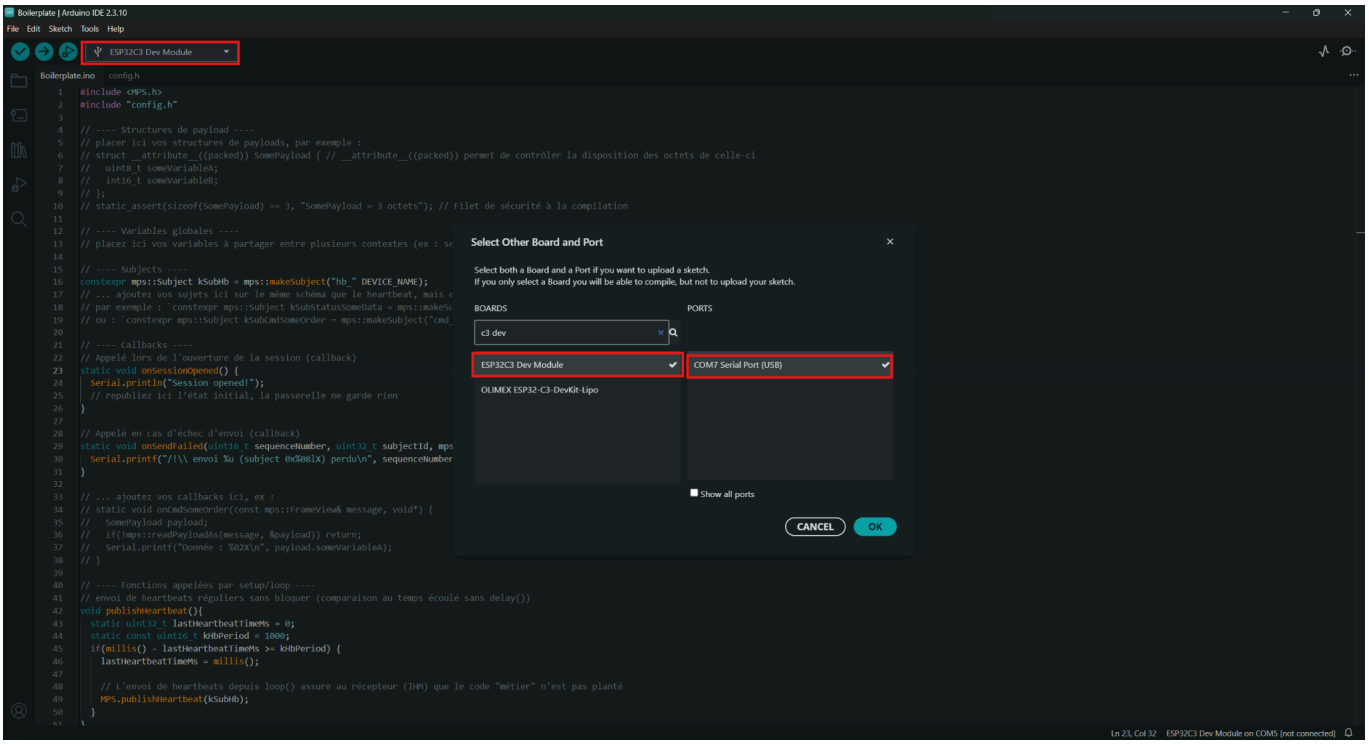
```
None
Passerelle : 90:DA:72:53:5D:08, canal 1  (à recopier dans le config.h de votre capteur)
```

Recopiez l’adresse dans l'onglet config.h (la MAC s'écrit octet par octet en hexadécimal) :

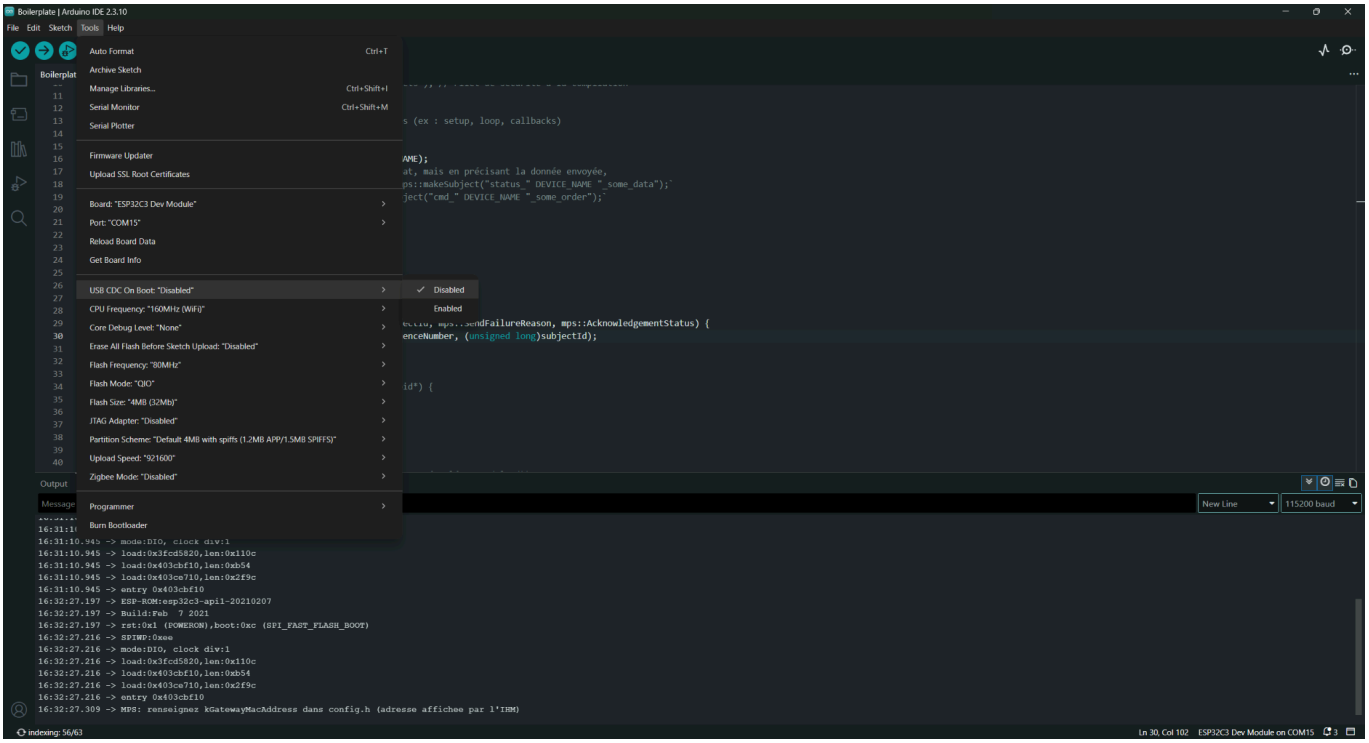
```
C/C++
static const uint8_t kGatewayMacAddress[6] = {0x90,0xDA,0x72,0x53,0x5D,0x08};
constexpr uint8_t kWifiChannel = 1;
#define DEVICE_NAME "sensor_a"    // exemple, sensor_<groupe>, minuscules, 16 caractères au plus
```

Tant que la valeur de kGatewayMacAddress vaut {0,0,0,0,0,0}, le croquis refuse de démarrer et le dit sur le moniteur série.

Sélectionnez le bon type de carte et le bon port (identifiez le en connectant votre carte alors que vous êtes déjà dans l’onglet de modification du port :



Désactivez également le "CDC On Boot" depuis Tools > USB CDC On Boot > Disabled :




Téléversez (→) tel quel, ouvrez le moniteur série à 115200 bauds. Vous devez voir :

```
None
00:07:09.185 -> [MPS] demarre : nom=sensor_boiler canal=1 passerelle=90:DA:72:53:5D:08
00:07:09.185 -> [MPS] enregistre : node_id=1 max_payload=238 keepalive=5s
00:07:09.185 -> Session opened!
```

Côté IHM (à lancer avant d'allumer le capteur, sinon appuyer sur "reset" ou le re-brancher), celle-ci doit afficher :

```
None
Appareil en ligne : sensor_a (identifiant 1, 58:8C:81:B9:39:04)
```

# Création d’un capteur connecté

Mise en route

## 3. DÉCLARATION DES SUJETS ET DU PAYLOAD

Un subject est le nom d'une donnée : <type>_<appareil>_<donnée>, en minuscules. Ici, un compteur publié par le capteur, et une commande pour le remettre à zéro.

```C/C++
// ---- Structures de payloads ----
struct __attribute__((packed)) CounterPayload {
  uint32_t value;
};
static_assert(sizeof(CounterPayload) == 4, "CounterPayload = 4 octets");

// ---- Variables globales ----
static CounterPayload counter = {0};

// ---- Subjects ----
constexpr mps::Subject kSubHb = mps::makeSubject("hb_" DEVICE_NAME); // déjà présent
constexpr mps::Subject kSubStatusCounter = mps::makeSubject("status_" DEVICE_NAME "_counter");
constexpr mps::Subject kSubCmdResetCounter = mps::makeSubject("cmd_" DEVICE_NAME "_reset_counter");
```

- **packed** : pas d'octets de bourrage, la structure fait exactement la taille de ses champs.

- **static_assert** : si la taille change par erreur, ça ne compile plus. L'IHM lit les mêmes octets dans le même ordre : le format est un contrat entre le capteur et la tuile.

- Un nom de subject invalide (majuscule, double ‘_’, …) donne une erreur de compilation qui cite invalidSubjectName_see_spec_3_2.

## 4. PUBLIER SANS BLOQUER

Dans loop(), après MPS.loop() et publishHeartbeat() :

```C/C++
static uint32_t lastCounterTimeMs = 0;
if (millis() - lastCounterTimeMs >= 2000) {
  lastCounterTimeMs = millis();
  counter.value++;
  MPS.publishValue(kSubStatusCounter, counter);
}
```

- jamais de delay() dans loop(). MPS.loop() doit tourner au moins toutes les 10 ms, sinon les acquittements se perdent et la session tombe. On compare millis() à la dernière date d'envoi, comme pour le heartbeat.

- **publishValue** renvoie mps::ErrorCode::NotRegistered tant que la session n'est pas ouverte : rien n'est envoyé, ce n'est pas une erreur grave.

# Création d’un capteur connecté

Mise en route

## 5. REPUBLIER L’ÉTAT A L’OUVERTURE DE LA SESSION

La passerelle ne garde rien. Quand l'IHM ou la passerelle redémarre, elle ne connaît plus votre valeur. Republiez-la dans le callback déjà prévu :

```
C/C++
static void onSessionOpened() {
  Serial.println("Session opened!");
  MPS.publishValue(kSubStatusCounter, counter);
}
```

## 6. RECEVOIR UNE COMMANDE

Implémentez la commande “onCmdReset” :

```
C/C++
// ---- Callbacks ----
static void onCmdReset(const mps::FrameView& message, void*) {
  if (message.payloadLength != 0) return;          // ce format n'a pas de payload
  counter.value = 0;
  MPS.publishValue(kSubStatusCounter, counter);    // l'IHM voit le résultat réel
}
```

Et dans setup(), ajoutez le subject et son callback avant MPS.begin(…) :

```
C/C++
MPS.on(kSubCmdReset, onCmdReset);
```

Pour une commande avec des données, utilisez mps::readPayloadAs(message, &payload) et ignorez le message s'il renvoie false (mauvaise taille). Le handler doit rester court : pas d'attente dedans.

Pourquoi republier le status après la commande ? L'acquittement prouve seulement que le message est arrivé. Seul le “status_” prouve que la commande a été appliquée : l'IHM ne doit mettre à jour son affichage qu'en le recevant.

# Création d'un capteur connecté

Mise en route

## 7. AFFICHER DANS L'IHM (GODOT)

Partez de la tuile boilerplate : dupliquez son dossier dans res://tiles/, puis mettez son DEVICE_NAME à la même valeur que dans config.h. Telle quelle, elle affiche déjà le node_id du capteur, son état de connexion et fait clignoter l'icône à chaque heartbeat. Testez-la avant d'ajouter quoi que ce soit.

Ajoutez ensuite à la scène un Label CounterLabel et un Button ResetButton, marqués « nom unique » (%), puis complétez le script :

Python

```
# ---- CONSTANTES ---- (à la suite de SUB_HB)
const SUB_STATUS_COUNTER = "status_" + DEVICE_NAME + "_counter"
const SUB_CMD_RESET = "cmd_" + DEVICE_NAME + "_reset"


func _ready() -> void:
    super()                    # obligatoire : c'est lui qui abonne la tuile
    %ResetButton.pressed.connect(func(): publish(SUB_CMD_RESET, PackedByteArray(), true))


func get_subscriptions() -> PackedStringArray:
    var subscriptions: PackedStringArray = []
    subscriptions.append(SUB_HB)
    subscriptions.append(SUB_STATUS_COUNTER)   # sans cette ligne, on_message ne reçoit rien
    return subscriptions


func on_message(subject_name: String, source_id: int, payload: PackedByteArray) -> void:
    if subject_name == SUB_HB: _on_hb(source_id)
    if source_id != _node_id: return
    if subject_name == SUB_STATUS_COUNTER:
        if payload.size() != 4: return          # le contrat : 4 octets, sinon on ignore
        %CounterLabel.text = "Compteur : %d" % payload.decode_u32(0)
```

Le bouton n'écrit pas 0 dans le label : il attend le "status_" renvoyé par le capteur.

Vérification attendue : le compteur avance toutes les 2s, et le bouton le remet à 0.